

Lab 1 - Modelling and Identification

Course notes – 2025/26 labs

Contents

1 Lab handout - objectives and tasks	1
2 Devkit guide - setup and usage	3
3 Devkit scripts - parameters and model wiring	4
4 Simulink models present	5
5 Report cover page template	6
6 Connections to lecture material	6
7 Open questions / unclear points	7

Source files:

- 25_26_AR_Lab1.pdf (7 pages) - "Laboratory Guide 1: Modeling and Identification of the Parrot AR.Drone", UAVs, MEAer, IST Lisboa, 2025/26 First Semester, September 2025.
- DevKit_Lab1/ARDroneSimulinkDevKit_V1.1/guide.pdf (3 pages) - "ARDrone Simulink Development Kit", v1.0, Sep 2013, by David Escobar Sanabria (U. Minnesota) and Pieter J. Mosterman (MathWorks).
- All .m scripts under DevKit_Lab1/ARDroneSimulinkDevKit_V1.1/ (start_here.m, lib/getWaypoints.m, lib/setupARModel.m, simulation/setupBaseModel.m, simulation/setupHoverSim.m, simulation/setupWPTrackingSim.m, wifiControl/setupHover.m, wifiControl/setupWPTracking.m).
- 25_26_UAVs_Lab_report_cover_page.docx (shared template for all 3 labs).

1 Lab handout - objectives and tasks

Objectives (handout §1.1): (1) kinematic/dynamic modelling of a quadrotor, (2) linearization about hover, (3) analysis of inner/outer control loops for height and vertical speed, (4) identification of the height closed-loop system, (5) identification of the pitch-angle closed-loop system.

Logistics (§1.2): questions are marked (T) theoretical (solve outside lab) or (L) lab (needs experimental data). Deliverable per group: a zip with (a) a single-column PDF report, **max 15 pages**, and (b) the .m script files, uploaded via Fenix. Must

use the official cover page as front page (see below). §1.3 is a standard academic-integrity clause (work must be original).

§2 Modelling (all (T) questions) - this section restates the rigid-body model from lectures Ch1/Ch2 with lab-specific notation (uses $\{I\}$ =NED frame explicitly, position $p = [x, y, z]$, velocity $v = [u, v, w]$ in body frame, Euler angles $\lambda = [\phi, \theta, \psi]$ with $\theta \in] - \pi/2, \pi/2[$, angular velocity $\omega = [p, q, r]$) and the same kinematics/dynamics equations as .../lectures-22-23/notes/01-foundations.pdf: $\dot{p} = R(\lambda)v$, $\dot{\lambda} = Q(\lambda)\omega$, $m\dot{v} = -mS(\omega)v + f$, $J\dot{\omega} = -S(\omega)J\omega + n$.

The AR.Drone-specific part: 4 rotors with individual thrusts f_i and torques $n_i = c \cdot f_i$ (c = a scalar drag/thrust proportionality constant, same role as c_Q/c_T in the lecture notes), two counter-rotating pairs. Exercises walk through:

- **2.1:** derive $f = M \cdot f_g + N \cdot f_T$ and $n = P \cdot f_T$, i.e. decompose total force/torque into gravity contribution (M) and rotor-thrust contribution (N, P) - this is the lab's version of the lecture's mixer-matrix derivation (.../lectures-22-23/notes/01-foundations.pdf §Ch2 actuation chain), but phrased with an explicit gravity term folded in rather than added afterward.
- **2.2:** redo the mixer derivation for a **45°-rotated body frame** (x_B axis midway between rotor arms) - directly the "× configuration" exercise that the lecture slides (Ch2 p.13) left open and unsolved; this lab handout supplies the concrete version of that exercise.
- **2.3:** define control input $u = [T; n]$ via $L \cdot f_T = u$, discuss usefulness of the linear transform, and derive the inverse $f_T = L^{-1}u$ - i.e. rotor-thrust allocation from desired thrust/torque, needed to actually command the AR.Drone.
- **2.4:** derive full 12-state nonlinear dynamics in two alternative state parametrizations, $x = [{}^B p, v, \lambda, \omega]$ (position expressed in **body** frame, ${}^B p = R^T p$) and $x_1 = [p, \dot{p}, \lambda, \omega]$ (position and its inertial-frame derivative) - $g(x, u)$ and $g_1(x_1, u)$.
- **2.5:** find the hover equilibrium (x_0, u_0) at arbitrary position p_0 and yaw ψ_0 - expect $T_0 = mg$, $\phi_0 = \theta_0 = 0$, all velocities/rates zero (matches the lecture's hover special case of differential flatness).
- **2.6-2.7:** linearize around hover to get $\dot{\tilde{x}} = A\tilde{x} + B\tilde{u}$ (12×12 A , 12×4 B), then derive the six transfer functions $G_\phi(s) = \Phi/N_x$, $G_\theta(s) = \Theta/N_y$, $G_\psi(s) = \Psi/N_z$, $G_x(s) = X/N_y$, $G_y(s) = Y/N_x$, $G_z(s) = Z/T$ (assuming diagonal J) - this is the lab's concrete instantiation of the small-angle linearization sketched in .../lectures-22-23/notes/02-control.pdf (Ch3, $\ddot{p} \approx [-g\theta, g\phi, 0]$).
- **2.8:** relate $G_\phi \leftrightarrow G_y$ and $G_\theta \leftrightarrow G_x$ (physical meaning: pitch drives x-motion, roll drives y-motion - the classic cross-coupling noted in the Ch3 lecture notes) and discuss the effect of different yaw equilibria ψ_0 .
- **2.9-2.10:** two equivalent height-control block diagrams (Fig. 2: nested inner-rate/outer-position PD structure with an explicit $1/(ms)$ mass-integrator and derivative feedback k_d ; Fig. 3: a single PD compensator $k_p + k_d s$ in series) - asked to compare them and sketch a root-locus in k_p for fixed $k_d > 0$. These map directly onto the Ch3 lecture material on root-locus/loop-shaping for the position/height loop.

§3 Identification of closed-loop height dynamics - (L) questions:

- **3.1** (simulation): modify ARDroneHoverSim.slx (save as ARDroneHover-SimHeight.slx) to get step responses for 4 proportional-gain values $k_p \in \{0.5, 1.0, 2.0, 3.0\}$ with correspondingly *decreasing* step sizes $\{1.0, 0.8, 0.3, 0.2\}$ m (rationale to discuss: larger gain $\rightarrow$ more aggressive response $\rightarrow$ smaller step needed to stay within safe/linear operating range or avoid saturation/overshoot into the ceiling or floor).
- **3.2** (experiment): repeat physically on the real drone via Wi-Fi (ARDroneHover.slx $\rightarrow$ save as ARDroneHoverHeight.slx), following a strict safety procedure: Fly switch = land, Commands = disabled, take off, stabilize at 0.75 m reference, enable commands, wait ≥ 2 s, then apply the cumulative step.
- **3.3**: use MATLAB's System Identification Toolbox to fit a closed-loop transfer function (matching the Fig. 2 structure) to the experimental step data.
- **3.4**: plot pole/zero maps per gain value, compare against the theoretical root-locus (Q2.10), discuss theory-vs-simulation-vs-experiment mismatches. Explicit **suggestion given in the handout**: model communication delay via a **Padé approximation** in the root-locus - a concrete, gradeable technique not covered explicitly in the lecture slides (which discuss delay only via phase-margin bounds, $\tau_c = PM/\omega_c$, in [../lectures-22-23/notes/02-control.pdf](#)).

§4 Identification of closed-loop pitch dynamics - (L) questions:

- **4.1**: obtain an experimental Bode amplitude diagram for pitch by injecting sinusoidal references, $\omega \in [1, 20]$ rad/s, amplitude 0.2 at low frequencies (amplitude presumably reduced at higher frequencies to avoid actuator saturation, though the handout doesn't spell out the schedule beyond "maximum amplitude 0.2 for the lower frequencies").
- **4.2**: fit a **2nd-order LTI model with a minimum-phase zero** to the measured amplitude response using MATLAB's nlinfit.
- **4.3**: apply a pulse-like pitch reference ($0 \rightarrow 0.2$ rad at $t = 15$ s, back to 0 at $t = 18$ s) and cross-check the response against both the step (parts 3) and frequency-response (part 4.1-4.2) characterizations for consistency.

2 Devkit guide - setup and usage

The devkit (ARDrone Simulink Development Kit v1.0, 2013, U. Minnesota/MathWorks) requires **MATLAB/Simulink R2013b** and **Real-Time Windows Target** (rtwintgt -setup), targeting a **Parrot AR.Drone 2.0** with the protective indoor hull.

Two core library blocks in lib/ARBlocks.slx:

- **ARDrone Simulation Block**: a vehicle-dynamics model obtained via system identification (i.e. this is the identified closed-loop/open-loop model referenced throughout the lab handout) - used to validate algorithms before real flight.
- **ARDrone Wi-Fi Block**: real interface to the physical drone (send commands / read states), built on Real-Time Windows Target for periodic execution, used for actual experiments.

Both blocks share the same I/O interface, so a controller built against the simulation block transfers directly to the Wi-Fi block.

Three worked examples, each with a simulation and a Wi-Fi-control variant:

1. **Waypoint tracking** - tracks a list of $(X_e, Y_e, \text{height, heading, waiting-time})$ waypoints (edited in `lib/getWaypoints.m`). Wi-Fi workflow: connect to drone's Wi-Fi network → edit waypoints → run `wifiControl/setupWPTracking.m` → build the Simulink model (Ctrl+B) → verify fly/enable reference commands/emergency stop inputs are 0 → connect to target and run → toggle `fly=1` to take off, then enable reference commands=1 to start the controller (heading zeroed at that moment).
2. **Hovering / position control** - simplified case with constant position/heading references; setup via `setupHoverSim.m` (sim) / `setupHover.m` (Wi-Fi), models `ARDroneHoverSim.slx` / `ARDroneHover.slx`.
3. **Baseline simulation** (`ARDroneBaseSim.slx`) - open-loop response to manual roll/pitch/yaw-rate/vertical-velocity commands; outputs body-frame velocities, orientation, height, battery level, drone state; three manual switches for take off/land/enable-commands/emergency-motor-cutoff.

Video streaming: front camera accessible via `ffmpeg/ffplay` at `http://192.168.1.1:5555` (raw h264 stream) - e.g. `ffplay -f h264 -i http://192.168.1.1:5555` to view live, or `ffmpeg ... -f image2 -vf fps=10 out%d.png` to dump frames. (192.168.1.1 is the AR.Drone's own Wi-Fi-AP default gateway address - confirms the drone hosts its own Wi-Fi network that the host computer joins directly, no router involved.)

3 Devkit scripts - parameters and model wiring

Script	Role	Notable details
<code>start_here.m</code>	Interactive menu entry point. Clears workspace, prompts "(1) Simulation / (2) Wi-Fi control", then a sub-menu (Baseline / Hover / Waypoint tracking), cds into the right folder and runs the matching setup script, then <code>cd .. back</code> .	Wi-Fi branch has commented-out <code>rtwbuild(...)</code> calls - actual real-time build is done manually (Ctrl+B in Simulink) per the guide's instructions, not automatically by this script.
<code>lib/getWaypoints.m</code>	Returns an 11-column matrix of $[X_e; Y_e; h; \text{heading}; \text{waitTime}]$ waypoints (despite the header comment describing only 4 fields - a minor doc/code mismatch, see Open Questions).	Default demo path: hover at origin (1 m) → shuttle to $(-1.5, 0)$ and $(1.5, 0)$ → bump height to 1.8 m → settle back at 1 m, with 5 s waits at each "parked" point. Meant to be hand-edited by students per experiment.

Script	Role	Notable details
lib/ setupARModel.m	Loads 6 pre-identified linear state-space models from .mat files: ssRoll, ssRoll2V, ssPitch, ssPitch2U, ssYaw, ssH (height).	These are exactly the "system identification" models the guide refers to as backing the ARDrone Simulation Block - i.e. already-identified roll/pitch/yaw/height dynamics ship with the kit; Lab 1's Q3/Q4 asks students to <i>re-identify</i> the height and pitch closed-loop dynamics themselves and presumably compare against these bundled models.
simulation/ setupBaseModel.m	Sets FMS.Ts = 0.065 s (control-law sample time), timeDelay = 4·Ts = 0.26 s (communication delay), simDT = 0.005 s (numerical integration step), loads setupARModel, getWaypoints, opens ARDroneBaseSim.	The 4·Ts delay figure is a concrete number for the Padé-approximation suggestion in handout Q3.4.
simulation/ setupHoverSim.m	Same pattern but FMS.Ts = 0.03 s (commented-out alternative 0.065), opens ARDroneHoverSim.	Faster nominal sample time for the hover/position-control case than the baseline/waypoint case.
simulation/ setupWPTrackingSim.m	FMS.Ts = 0.065 s, timeDelay = 4·Ts, opens ARDroneWPTrackingSim.	Same timing as baseline.
wifiControl/ setupHover.m	sampleTime = 0.03 s (commented 0.065), adds ../lib to path, opens ARDroneHover. Carries an explicit warning comment : position estimate is inaccurate because it integrates a noisy onboard velocity estimate (itself from optical flow), so drift should be expected.	This is the course's own hint that the AR.Drone's dead-reckoned position is unreliable - directly relevant to why Lab 2 (estimation) exists and to the sensors notes' point that the AR.Drone has no GPS.
wifiControl/ setupWPTrackingDemo.m	sampleTime = 0.065 s ("In demo mode, the ARDrone sends data via UDP every 0.065 s" - confirms UDP as the telemetry transport), loads waypoints, opens ARDroneWPTracking. Same position-accuracy warning as above.	Confirms UDP, not TCP, for the Wi-Fi telemetry/command link.

4 Simulink models present

All under DevKit_Lab1/ARDroneSimulinkDevKit_V1.1/:

- lib/ARBlocks.slx - shared library (ARDrone Simulation Block + ARDrone Wi-Fi Block), imported by every other model.

- simulation/ARDroneBaseSim.slx - open-loop baseline simulation (manual roll/pitch/yaw-rate/vertical-speed inputs).
- simulation/ARDroneHoverSim.slx - closed-loop hover/position-control simulation; **this is the model Lab 1 §3.1 asks students to copy and modify** for the height step-response identification.
- simulation/ARDroneWPTrackingSim.slx - closed-loop waypoint-tracking simulation.
- wifiControl/ARDroneHover.slx - real-drone hover/position control over Wi-Fi; **copied/modified for Lab 1 §3.2/§4** (height and pitch identification experiments).
- wifiControl/ARDroneWPTracking.slx - real-drone waypoint tracking over Wi-Fi.

Note: the lab handout instructs saving modified copies under new names (e.g. ARDroneHoverSimHeight.slx, ARDroneHoverHeight.slx, ARDroneHoverPitch.slx) - none of these derived files exist yet in this devkit copy; they are expected to be created by the students during the lab.

5 Report cover page template

25_26_UAVs_Lab_report_cover_page.docx (extracted text): "Instituto Superior Técnico / MEAer / Unmanned Aerial Vehicles / 2025/2026 – First Semester / **Laboratory ?**" then a "**Group ?**" block with 3 rows of N.º: ? Name: ? Signature: (student number, name, signature - one row per group member) and a final Instructor: ? field. Applies identically to Labs 1, 2, and 3 - only the "Laboratory ?" number changes per submission.

6 Connections to lecture material

- The rigid-body kinematics/dynamics equations in handout §2 are a direct restatement of [../../../../lectures-22-23/notes/01-foundations.pdf](#) (Ch1), down to matching notation ($S(\cdot)$, $Q(\lambda)$, Z-Y-X Euler angles).
- The $f = Mf_g + Nf_T$, $n = Pf_T$ decomposition (Q2.1) and the $u = Lf_T$ allocation (Q2.3) are the lab's concrete version of the Ch2 mixer-matrix derivation; Q2.2's 45°-rotated-frame case is literally the "× configuration" exercise the Ch2 lecture slides left unsolved (see [../../../../lectures-22-23/COURSE_NOTES.pdf](#) gotchas list) - **this lab handout is where that exercise actually gets worked out.**
- The hover-equilibrium and linearization exercises (Q2.5-Q2.8) instantiate the small-angle linearization from [../../../../lectures-22-23/notes/02-control.pdf](#) (Ch3) with explicit transfer functions per axis.
- The two height-control block diagrams (Q2.9, Figs. 2-3) and the root-locus/gain-tuning discussion (Q2.10, §3) are a direct hands-on application of the Ch3 root-locus/loop-shaping toolkit ([../../../../lectures-22-23/notes/02-control.pdf](#)), now closing the loop with **real experimental identification** rather than a purely theoretical design.

- The Padé-approximation suggestion for modelling the communication delay (Q3.4) extends the Ch3 lecture’s delay-margin treatment ($\tau_c = PM/\omega_c$) into an explicit root-locus construction - a technique not spelled out in the lecture notes themselves.
- Frequency-response identification (§4, Bode fitting via `nlinfit`) parallels the Bode/loop-shaping analysis tools from Ch3, but used here for *system identification* (fitting an unknown plant) rather than *controller design* (shaping a known loop).
- This lab is explicitly “L1 = Modelling and identification” in the course’s own lab-to-chapter mapping (see [../lectures-22-23/COURSE_NOTES.pdf](#), “Course $\leftrightarrow$ lab mapping”), i.e. it’s graded at weight 0.2 in the lab average $L = 0.2 \cdot L1 + 0.4 \cdot L2 + 0.4 \cdot L3$.

7 Open questions / unclear points

- `lib/getWaypoints.m`’s header comment describes a 4-field waypoint (position, heading, waiting time) but the actual code builds a 5-row matrix (X_e, Y_e, h , heading, waitTime - position split into 2 rows) - minor documentation/code mismatch in the vendor-supplied devkit, not something introduced by this course.
- The devkit guide and scripts are from 2013 (MATLAB R2013b, Real-Time Windows Target) while the lab is dated September 2025 - worth confirming with the instructor/TA whether a newer MATLAB/Simulink Real-Time toolchain is actually used in the 2025/26 labs, since R2013b is very outdated and Real-Time Windows Target was deprecated/renamed (now “Simulink Real-Time”) in later MATLAB releases.
- The exact numeric content of the pre-identified `.mat` state-space models (`ssRoll`, `ssPitch`, `ssH`, etc.) loaded by `setupARModel.m` was not inspected (binary `.mat` files) - these likely contain the “ground truth” identified models students are meant to compare their own Lab 1 identification results against, but their actual coefficients are unknown from this analysis.
- The handout’s Q4.1 sinusoidal test amplitude schedule (“maximum amplitude 0.2 for the lower frequencies”) is not fully specified for higher frequencies in $\omega \in [1, 20]$ rad/s - presumably reduced to avoid actuator saturation at higher frequencies, but the exact schedule is left to the student/TA to determine experimentally.
- The `.slx` files themselves are binary (zipped XML) and were not parsed in detail - block-level control-law structure (e.g. exact PD gains wired into `ARDroneHoverSim.slx`) is inferred only from the guide text and handout, not verified against the actual block diagrams.